MIP_METADATA_MODEL.md

Mainframe Intelligence Platform

Canonical Metadata Model

Version: 1.0

Status: Foundational Architecture

Classification: Core Platform Specification

Purpose

The Metadata Model is the canonical representation of a legacy system within MIP.

MIP does not reason directly on source code.

MIP reasons on metadata.

The metadata model transforms:

COBOL

JCL

Copybooks

DB2

VSAM

CICS

MQ

IMS

↓

Canonical Metadata

↓

Knowledge Graph

↓

Reasoning

↓

Modernization Intelligence

This document defines the enterprise-wide metadata language used throughout the platform.

Design Principles

Technology Agnostic

Metadata must not be tied to COBOL.

Future languages:

- Java
- C#
- Python
- PL/I
- RPG

must map into the same model.

Evidence Based

Every metadata element must reference source evidence.

Example:

```
Program Name = TS2AUTH

Evidence:
src/auth/TS2AUTH.cbl
Line 12
```

Graph First

All metadata must support graph construction.

Extensible

New entity types must be added without redesign.

Explainable

Every recommendation must be traceable to metadata.

Core Metadata Architecture

```
Repository
    |
    +-- Application
            |
            +-- Domain
                    |
                    +-- Capability
                            |
                            +-- Program
                            +-- Job
                            +-- Dataset
                            +-- Table
                            +-- Transaction
```

Entity Taxonomy

MIP uses three layers.

Layer 1

Business Entities

Layer 2

Technical Entities

Layer 3

Relationship Entities

Business Entities

These represent business meaning.

Application

Represents a business application.

Examples:

- Card Processing
- Core Banking
- Claims Management

Attributes

```
application_id
application_name
description
owner
criticality
created_at
updated_at
```

Domain

Represents a business domain.

Examples:

- Authorization
- Settlement
- Billing
- Customer Management

Attributes

```
domain_id
domain_name
description
application_id
```

Capability

Represents business capability.

Examples:

- Card Activation
- Balance Inquiry
- Payment Processing

Attributes

```
capability_id
capability_name
description
domain_id
```

Technical Entities

These are discovered from source code.

Program

Most important technical entity.

Attributes

```
program_id
program_name
program_type

language

path

loc

cyclomatic_complexity

called_program_count

copybook_count

table_count

```

```
dataset_count

created_at
updated_at
```

Program Types

```
ONLINE

BATCH

UTILITY

SERVICE

UNKNOWN
```

Job

Represents JCL job.

Attributes

```
job_id

job_name

step_count

schedule_type

description
```

Job Step

Represents execution step.

Attributes

```
step_id

job_id
```

```
step_name

sequence_number

program_name
```

Copybook

Represents reusable structures.

Attributes

```
copybook_id

copybook_name

path

field_count

record_count
```

Dataset

Represents batch data.

Attributes

```
dataset_id

dataset_name

dataset_type

retention

owner
```

Dataset Types

```
INPUT

OUTPUT
```

```
TEMPORARY

VSAM

GDG
```

Table

Represents DB2 object.

Attributes

```
table_id

table_name

schema_name

row_count

criticality
```

Column

Attributes

```
column_id

table_id

column_name

data_type

nullable
```

Transaction

Represents online transaction.

Examples:

- CICS Transaction
- Screen Flow
- API Call

Attributes

```
transaction_id

transaction_name

transaction_type

entry_program
```

VSAM File

Attributes

```
vsam_id

dataset_name

organization

key_length

record_length
```

Organizations

```
KSDS

ESDS

RRDS
```

Relationship Model

Relationships are first-class citizens.

CALLS

Program → Program

Example

```
AUTHDRV

CALLS

AUTHVAL
```

EXECUTES

Job → Program

USES_COPYBOOK

Program → Copybook

READS_TABLE

Program → Table

WRITES_TABLE

Program → Table

READS_DATASET

Program → Dataset

WRITES_DATASET

Program → Dataset

STARTS_TRANSACTION

Transaction → Program

PRODUCES

Program → Dataset

CONSUMES

Program → Dataset

BELONGS_TO_CAPABILITY

Program → Capability

BELONGS_TO_DOMAIN

Capability → Domain

Evidence Model

Every metadata record must contain evidence.

Attributes

```
evidence_id

source_file

source_type

line_start

line_end

extracted_by

confidence_score
```

Example

```
PROGRAM-ID AUTHDRV

File:
AUTHDRV.cbl

Lines:
10-10
```

Confidence Model

Every extracted fact receives confidence.

Range

```
0.0 - 1.0
```

Example

```
PROGRAM-ID

1.0

Capability Assignment

0.72
```

Metadata Quality Rules

Required:

Program

```
program_name
path
```

Required:

Job

```
job_name
```

Required:

Table

```
table_name
```

Invalid metadata should be quarantined.

Metadata Lifecycle

```
Discovery

↓

Extraction

↓

Validation

↓

Enrichment

↓

Graph Construction

↓

Reasoning

↓

Modernization
```

Metadata Queries

The model must support queries such as:

Which programs call TS2AUTH?

Which jobs execute TS2AUTH?

Which datasets are produced?

Which tables are updated?

Which capability owns this program?

Which modernization wave contains this capability?

Success Criteria

A modern engineer should be able to understand:

- What exists
- How it interacts
- What it impacts
- What capability it supports
- How it should be modernized

without reading COBOL source code.

MIP Principle

Source code is implementation.

Metadata is knowledge.

Knowledge drives modernization.

End of Document.